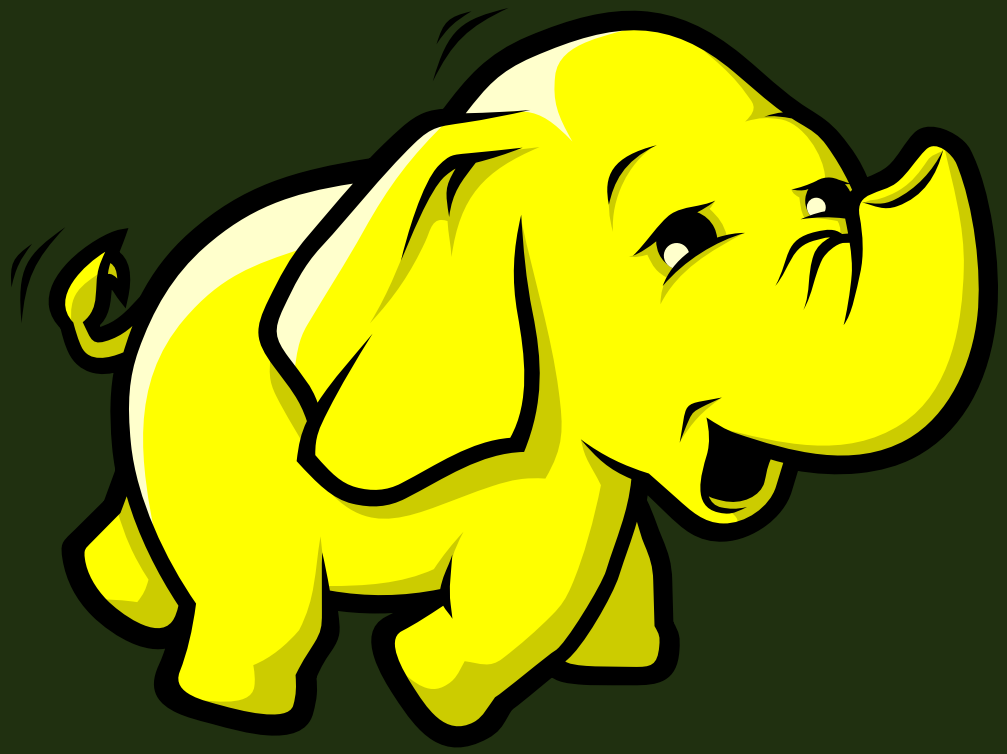


Hadoop Spark Million Songs Project

Park JaeHeung, Adam Michal Suszczewicz, Huang Mohan, Wang Yixiang, Zhou Zhaojiacheng

Table of contents

1. Project Setup
2. Drill SQL Queries Execution
3. Similar Artists Search
4. Prediction Year

1. Project Setup

Data Preparation

Mount and unmount data from school server to use HDF5 Files.
Download files starting with certain alphabet for practical use.

```
mkdir ECE472
sshfs -o port=2223,allow_other ece472@focs.ji.sjtu.edu.cn: ./ECE472
7z e ECE472/millionsong.iso data/A/* -o./Songs/A
umount ECE472
```

Download files starting with certain alphabet for practical use.

Run school server to handle entire 267 GB Data with 1,000,000 HDF5 files.

Compact HDF5 File

Compact countless HDF5 Files in one folder to one big HDF5 File.

Process to compact a million small H5 files in one folder into one single huge H5 file.

We used divide and conquer, compact data file for each alphabet.

```
allh5 = utils.get_all_files(maindir, ext='.h5')
HDF5.create_aggregate_file(output, expectedrows=len(allh5), summaryfile=False
print('Aggregate file created, we start filling it.')
h5 = HDF5.open_h5_file_append(output) # fill it
HDF5.fill_hdf5_aggregate_file(h5, allh5, summaryfile=False)
```

Covert HDF5 to Avro File

Convert compacted HDF5 File to Avro File Conversion of file from HDF5 to Avro reduces total file size significantly from 267 GB to 6.7GB, reducing size of file into less than 1 Percent.

```
schema = avro.schema.parse(schema_file.read())
fields = schema.fields
for field in fields:
getter_method = 'get_' + field.name
hdf5 = hdf5_getters.open_h5_file_read(hdf5_path)
num_songs = hdf5_getters.get_num_songs(hdf5)
writer = DataFileWriter(open(avro_path, "wb"), DatumWriter(), schema)
```

2. Drill SQL Queries Execution

Oldest & Latest Date Range

Oldest Song in Dataset is 1922, Latest Song in Dataset is 2011

```sql
SELECT MIN(year) AS OLDEST_SONG_YEAR
FROM AvroSongs.avro.`CompactAvro_*`
WHERE year IS NOT NULL AND year <> 0;

SELECT MAX(year) AS YOUNGEST_SONG_YEAR
FROM AvroSongs.avro.`CompactAvro_*`
WHERE year IS NOT NULL AND year <> 0;
```

```
+------------------+    +--------------------+
| OLDEST_SONG_YEAR |    | YOUNGEST_SONG_YEAR |
+------------------+    +--------------------+
| 1922             |    | 2011               |
+------------------+    +--------------------+
```

Hottest Short Song with High Energy and Low Tempo

Jingle Bell Rock

```sql
SELECT song_id, title, song_hotttnesss, duration, energy, tempo
FROM AvroSongs.avro.`CompactAvro_*`
WHERE song_hotttnesss IS NOT NULL
ORDER BY song_hotttnesss DESC, duration ASC, energy DESC, tempo ASC
LIMIT 5;
```

```
+--------------------+-----------------------------------+-----------------+
|      song_id       |                title               | song_hotttnesss |
+--------------------+-----------------------------------+-----------------+
| SONASKH12A58A77831 | Jingle Bell Rock                  | 1.0             |
| SOAVJBU12AAF3B370C | Rockin' Around The Christmas Tree | 1.0             |
| SOEWAKD12AB01860D5 | Holiday                           | 1.0             |
| SOAAXAK12A8C13C030 | Immigrant Song (Album Version)    | 1.0             |
| SOAXLDX12AC468DE36 | La Tablada                        | 1.0             |
+--------------------+-----------------------------------+-----------------+
```

Albums with most tracks

Greatest Hits

```sql
SELECT release AS ALBUM_NAME, COUNT(*) AS TRACK_COUNT
FROM AvroSongs.avro.`CompactAvro_*`
GROUP BY release
ORDER BY TRACK_COUNT DESC
LIMIT 5;
```

```
+-------------------------+-------------+
|       ALBUM_NAME        | TRACK_COUNT |
+-------------------------+-------------+
| Greatest Hits           | 2014        |
| Live                    | 1197        |
| The Collection          | 843         |
| The Ultimate Collection | 791         |
| The Very Best Of        | 741         |
+-------------------------+-------------+
```

Band with the Longest Song

Mystic Revelation of Rastafari

```sql
SELECT artist_name AS BAND_NAME, duration AS LONGEST_SONG_DURATION
FROM AvroSongs.avro.`CompactAvro_*`
ORDER BY duration DESC
LIMIT 5;
```

```
+--------------------------------+-----------------------+
|            BAND_NAME           | LONGEST_SONG_DURATION |
+--------------------------------+-----------------------+
| Mystic Revelation of Rastafari | 3034.9058             |
| Ustad Rashid Khan              | 3033.5996             |
| Galexis                        | 3033.4429             |
| Jean-Luc Godard                | 3032.7637             |
| Heiko Grauel                   | 3032.5808             |
+--------------------------------+-----------------------+
```

3. Similar Artists Search

Target & Outline

Our main goal was to search for similar artists using Breadth First Search (BFS) and compare the runtimes of MapReduce and Spark implementations.
BFS was chosen due to ease of parallelization which is preferred for distributed systems.

Main Logic

1. Load data from SQLite database.

$$< artistID > | < neighbor >$$

2. Initialize columns.

$$< artistID > | < neighbors > | < distance > | < state >$$

3. Run BFS for 5 iterations
   - Map
   - Reduce
4. Filter the top n artists with the smallest distance

Mapper

Algorithm 1 Pseudo code for the Mapper part

Require: (**Key**: artist, **Value**: (distance, neighbors, state))

```
result = []
if state == "discovered" then
        state = "processed"
    for each neighbor in neighbors do
            Append (neighbor, (distance + 1, [], "discovered")) to results
    end for
end if
Append (artist, (distance, neighbors, state)) to results
Emit(results)
```

Reducer

Algorithm 2 Pseudo code for the Reducer part

Require: node1 (**Key1**: artist, **Value1**: (distance, neighbors, state))
Require: node2 (**Key2**: artist, **Value2**: (distance, neighbors, state))

```
distance = min(dist1, dist2)
neighbors = union of neighbors1 and neighbors2
state = highest priority state (processed > discovered > undiscovered)
Emit (distance, neighbors, state)
```

Strategy

We tried to implement the algorithm in both MapReduce and Spark.

In order to improve speed, we also tried three methods in Spark to load data and initialize columns:

- Directly deal with the SQlite database using spark.
- Write a driver.sh to load data, initialize columns and run the spark code.
- Convert data into csv file **before running**, and directly use spark.

Performance

method	artist1	artist2	artist3	artist4	artist5
MapReduce	162	152	151	152	151
Spark	31	31	34	29	29
Shell+Spark	24	22	22	23	21
Shell	20	22	19	19	22

Table: Performance of different method

Spark is faster than MapReduce for around 8 times!

4. Prediction Year

Target & Outline

Our target in this part is to:

- Predict the year of song based on **content related to song itself**
- **Improve efficiency** for processing data and fit model.
- Apply models to **improve the model accuracy**.

The method we applied in this part including:

- Data cleaning
- EDA of dataset
- Dimension reduction method like PCA, random projection
- Regression method like linear regression, logistic regression and regression based on tree.

Data Cleaning

```
# clean dataset
df = df.drop("artist_hotttnesss", "release",
"song_hotttnesss", "song_id", "track_id", "artist_name",
"artist_id", "energy", "title", "danceability")

df = df.filter(df.year != 0)
df = df.filter(df.tempo != 0)

df = df.dropna()

df.show()
```

The origin data included missing years, empty variable and not related attributes like ID, release and artist name etc. We simply dropped them.

Data Cleaning

1. **Remove Unnecessary Columns:**
   - `artist_hotttnesss`
   - `release`
   - `song_hotttnesss`
   - ...
2. **Filter Invalid Data:**
   - Remove rows where `year` $= 0$
   - Remove rows where `tempo` $= 0$
3. **Handle Missing Values:**
   - Drop all rows with `NaN` values

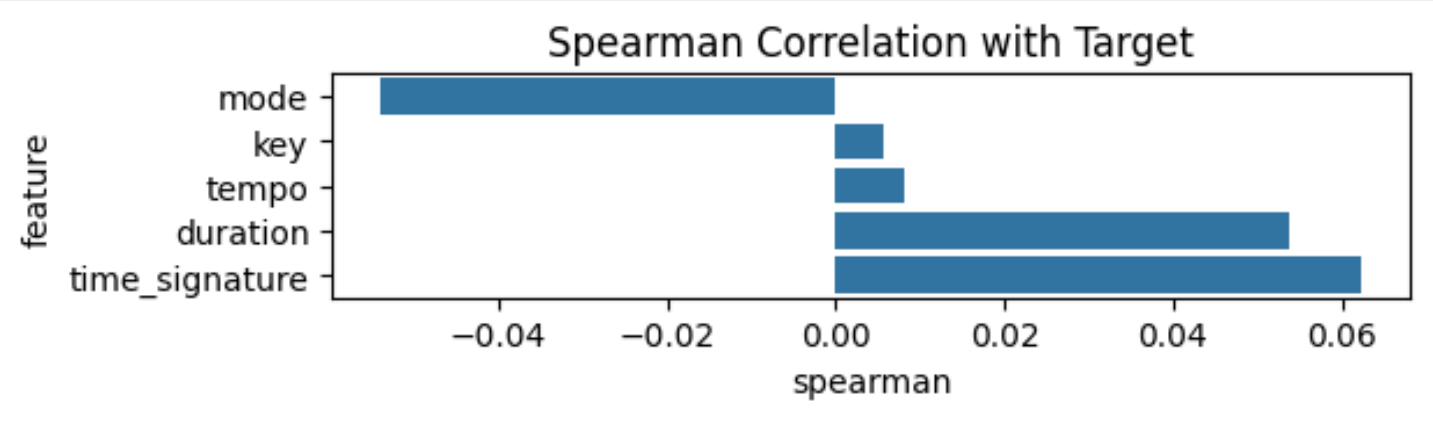
Sample Output

```
+----+-------+-------+-------+----+-------+-------+---+
|year|     bs|    bts|     ts|tsig|    dur|    tmp|...|
+----+-------+-------+-------+----+-------+-------+---+
|1996|[0.1...|[0.1...|[0.1...|   4| 244.61| 160.30|...|
|1994|[1.7...|[0.8...|[0.8...|   4| 453.93| 137.06|...|
|1994|[1.1...|[0.5...|[0.3...|   4|  82.81| 100.14|...|
|1957|[0.5...|[0.2...|[0.2...|   4| 562.60| 178.02|...|
|1998|[0.6...|[0.7...|[0.3...|   1|  36.21|  84.21|...|
|2008|[1.4...|[0.2...|[0.2...|   4| 263.31|  96.99|...|
|2001|[1.4...|[0.2...|[0.1...|   4| 216.79| 106.47|...|
|1987|[0.3...|[0.4...|[0.1...|   1| 284.58| 144.39|...|
|2001|[0.4...|[0.1...|[0.1...|   4| 100.10| 187.23|...|
|2008|[1.3...|[0.2...|[0.2...|   4| 273.08| 109.80|...|
+----+-------+-------+-------+----+-------+-------+---+
```

Above is a sample of cleaned data where the significant figures have been reduced for display purposes.

Spearman Correlation Analysis

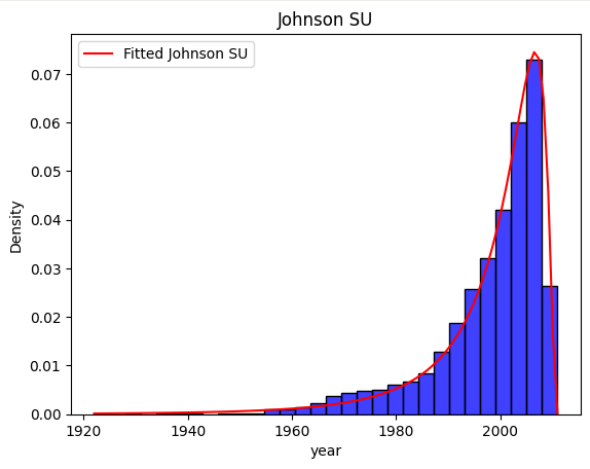
- Calculate the Spearman rank correlation between the variable and the target variable **year** and visualize these correlations using a bar chart.

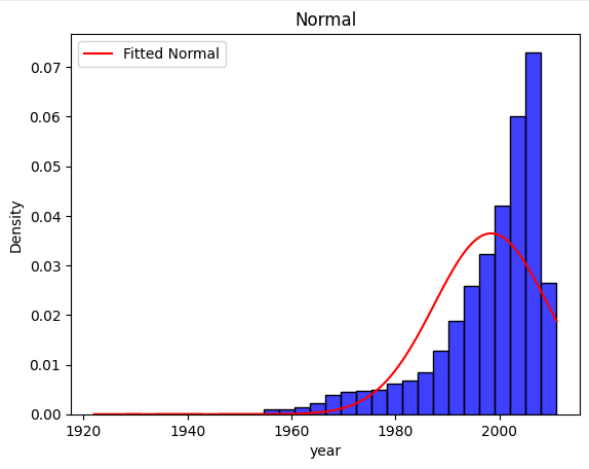


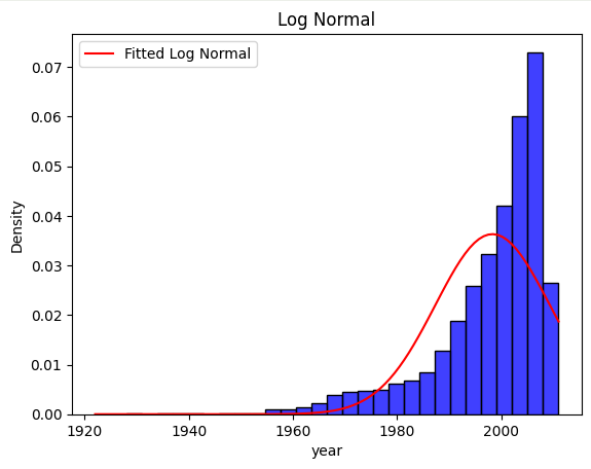
- We notice that the correlation is small, so it is unlikely that they have a linear relation with year.

Exploratory Data Analysis

- Since many dimension reduction method or regression method need data follow normal distribution, we need to find proper transformation method.
- Find data distribution for fitting them with different distribution model.

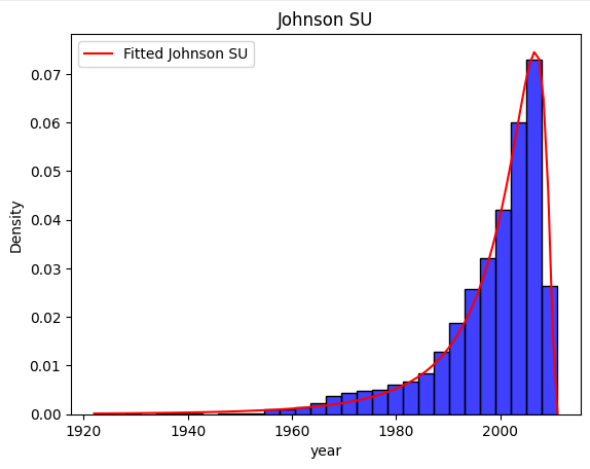


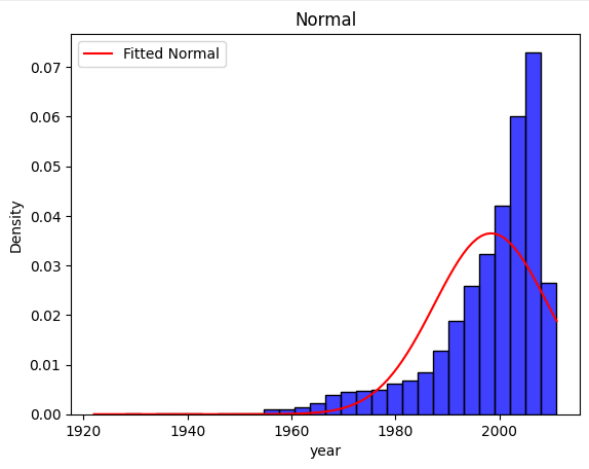


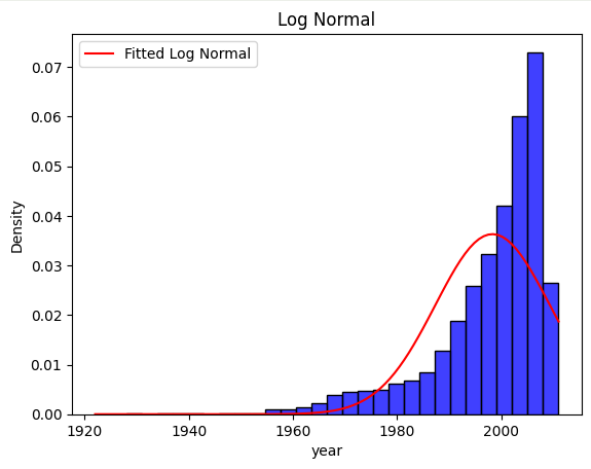


Exploratory Data Analysis

- Since many dimension reduction method or regression method need data follow normal distribution, we need to find proper transformation method.
- Find data distribution for fitting them with different distribution model.







- We notice that year fits the Johnson SU distribution best. This facilitates subsequent data analysis and modeling efforts.

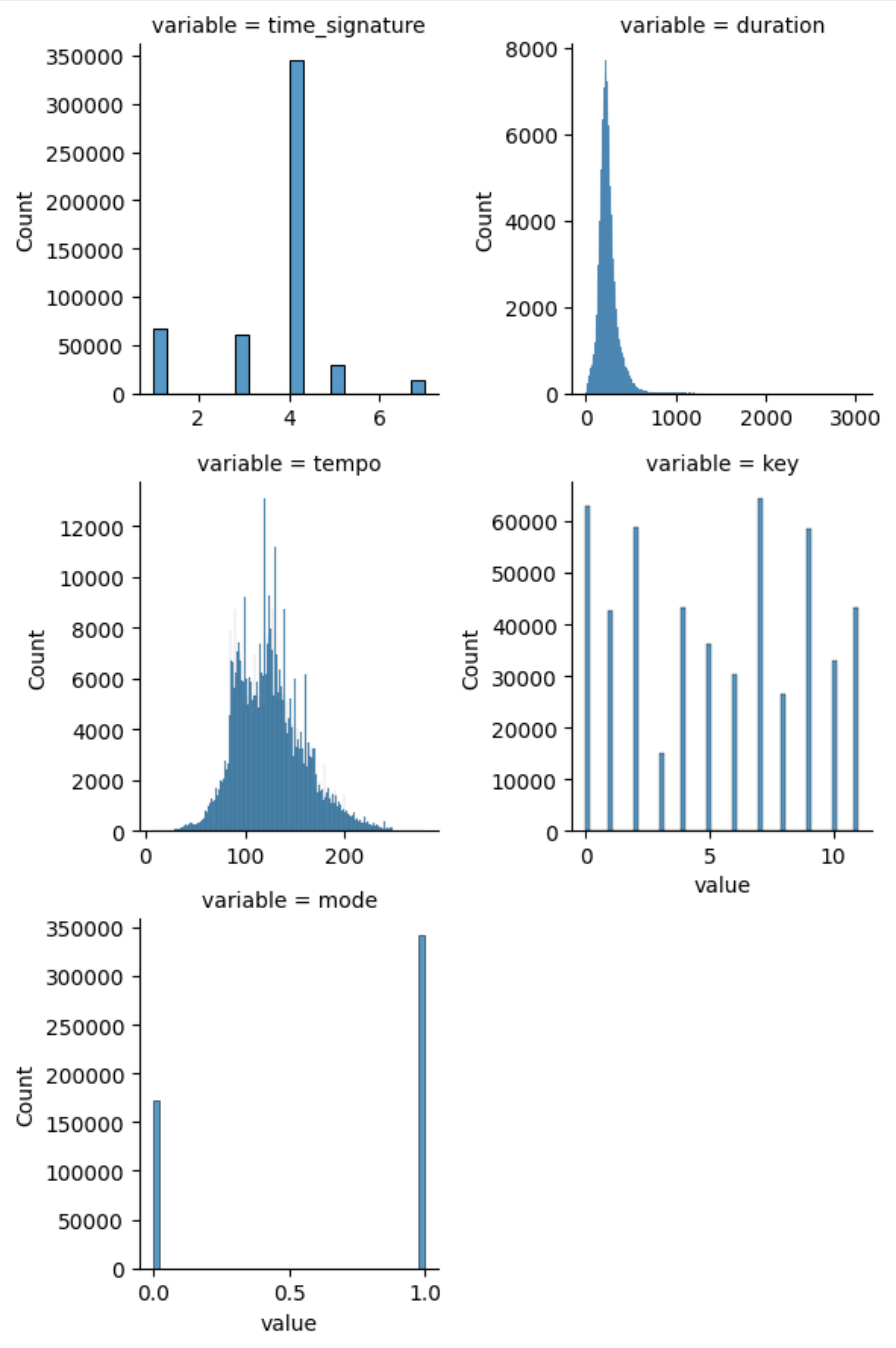
Normal Distribution Checking

- Then we check whether attributes follow the normal distribution

```
variable  p_value
0  time_signature      0.0
1        duration      0.0
2           tempo      0.0
3             key      0.0
4            mode      0.0
Any variable not normally distributed: True
```

- All variables have p values less than 0.01, hence none of them conform to the normal distribution.

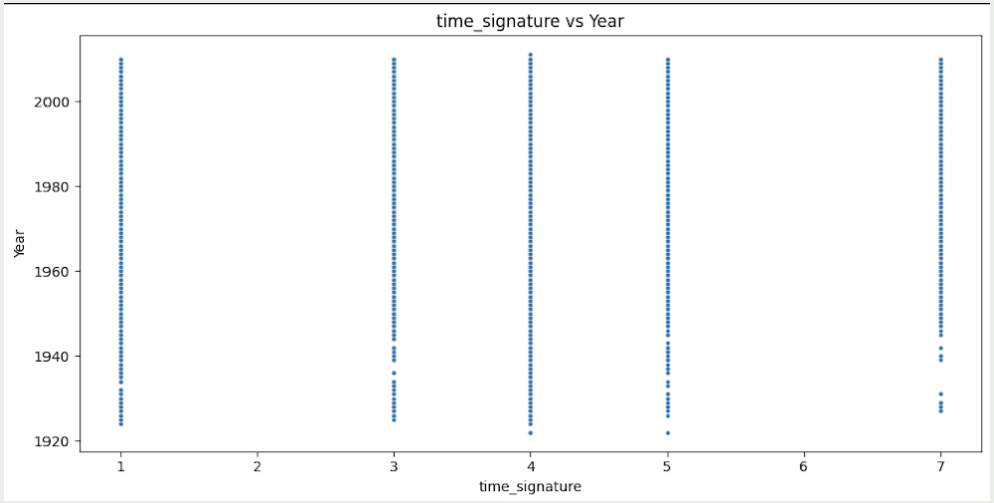
Exploratory Data Analysis

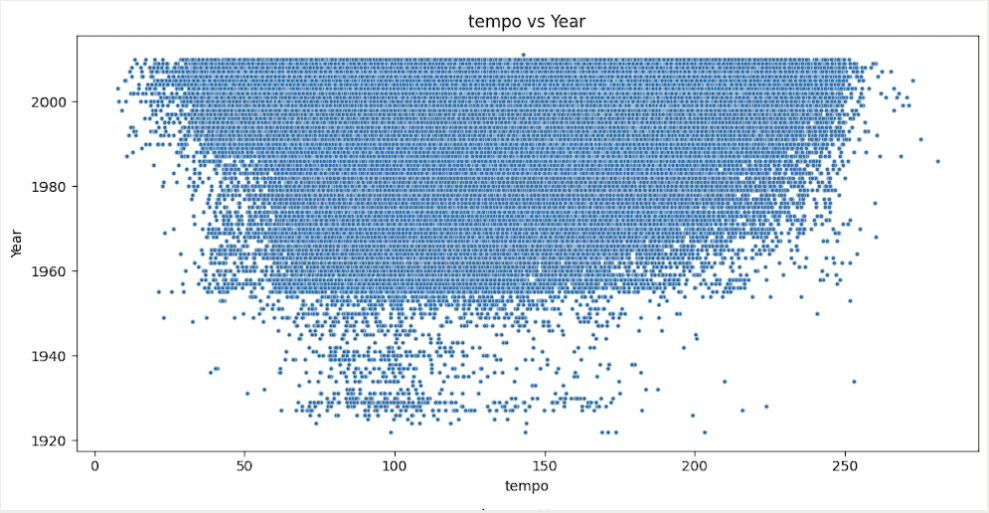


- The histogram shows that all attributes have some skew all
- Also, we found that some attributes are actually labels instead of number
- One-hot encode is applied to fix it

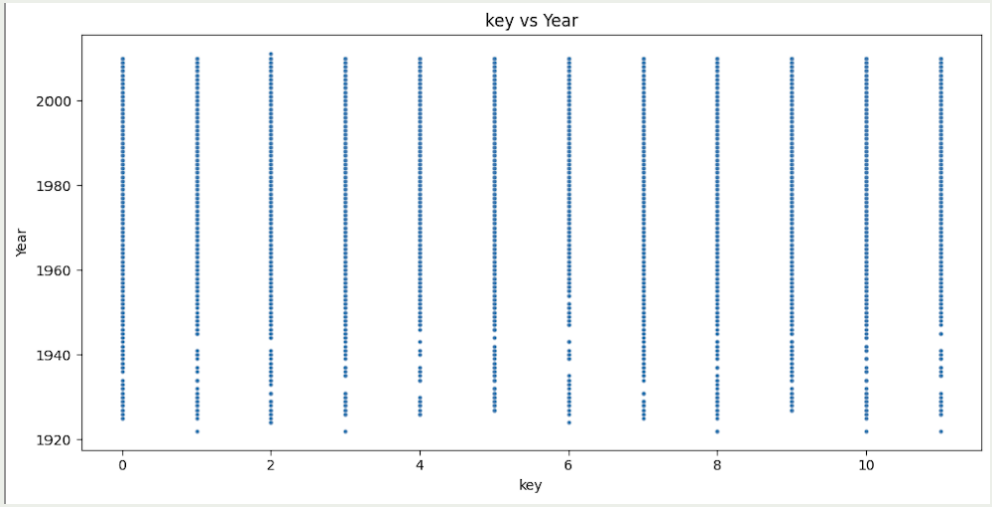
Exploratory Data Analysis

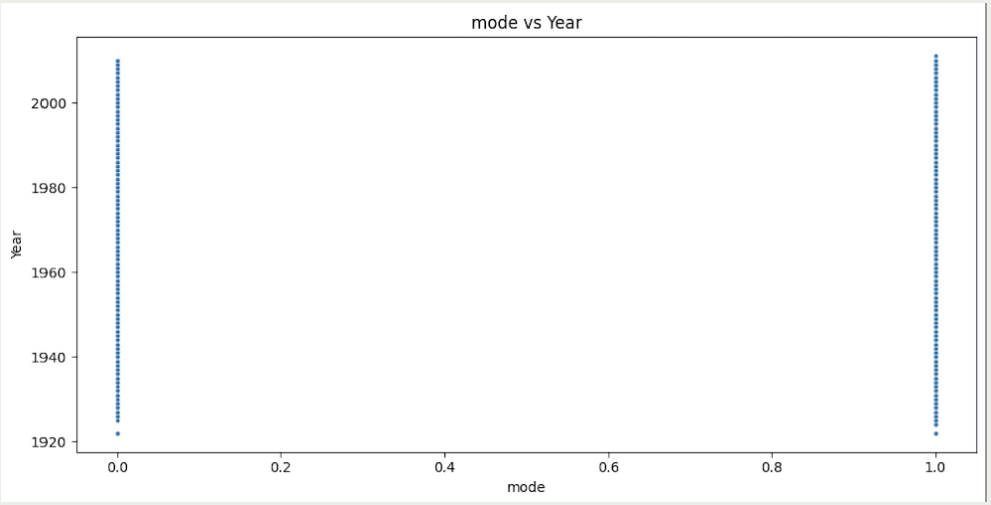
- Plot a pair scatter plot between variables and the year variable.
- The plots are as following

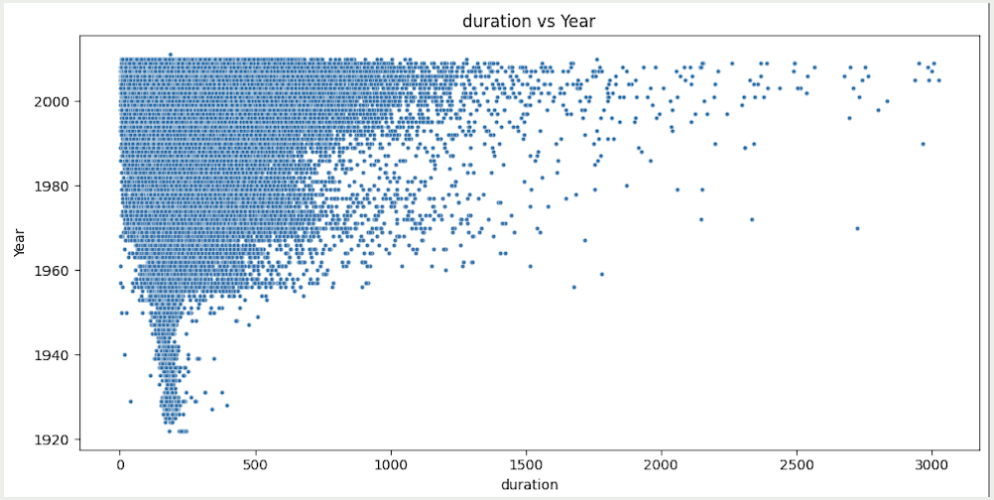




Exploratory Data Analysis

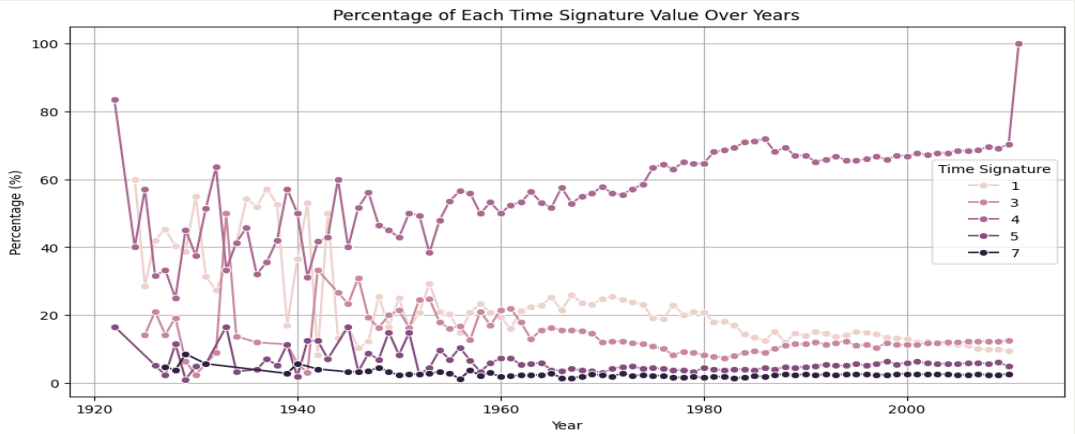


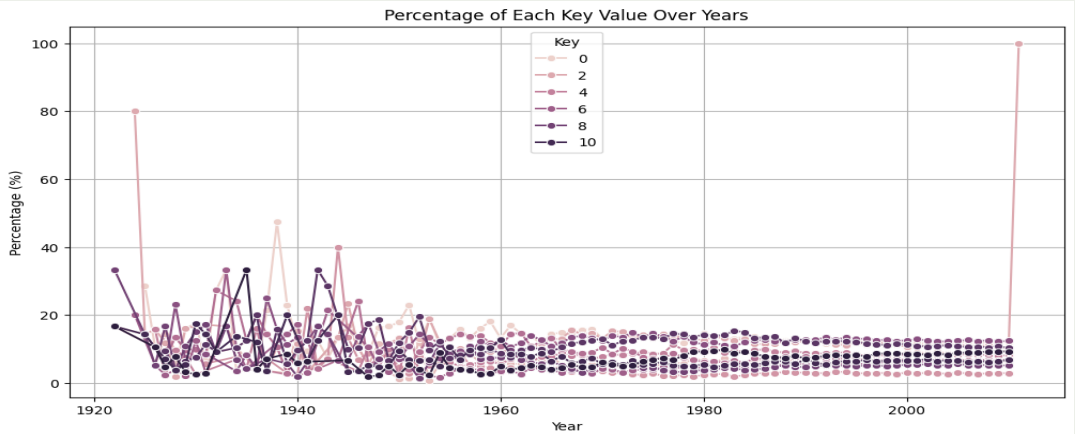


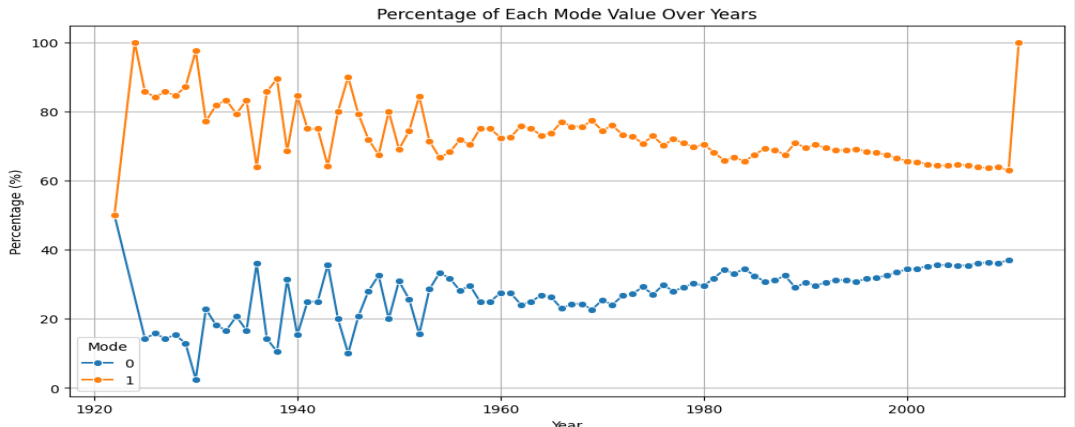


- These charts can help analyze the trend of each variable over time.

Exploratory Data Analysis







- We calculate the frequency of time signature, key and mode in each year
- All these variables change according to the year

Exploratory Data Analysis

- We also include data whose type is array
- We extract statistical features from array to
  - reduce data size as well as
  - fit array with different length

Exploratory Data Analysis

In this part, we successfully:

- Make the dataframe structure easier to process
- Decrease the data frame size for about 100 times
- Find distribution of dataset for further process
- Get insight on the regression strategy

Dimension Reduction Process

- There are three good ways to reduce data dimension: PCA, SVD and random projection
- Since the matrix is not sparse, we didn't apply SVD
- In this part we apply both of them and compared their results.
- In this project, we chose the number of principal components with an explanatory variance greater than 90%.

PCA

- PCA achieved reduce data from 15 dimensions to 3 dimensions while explaining 90% percent of variance.
- The runtime for PCA is about 3 minutes

Random Projection

- The random projection obtain similar results to PCA, three dimension explained more than 90% variance.
- The runtime for random projection is about 30–40 seconds

Results

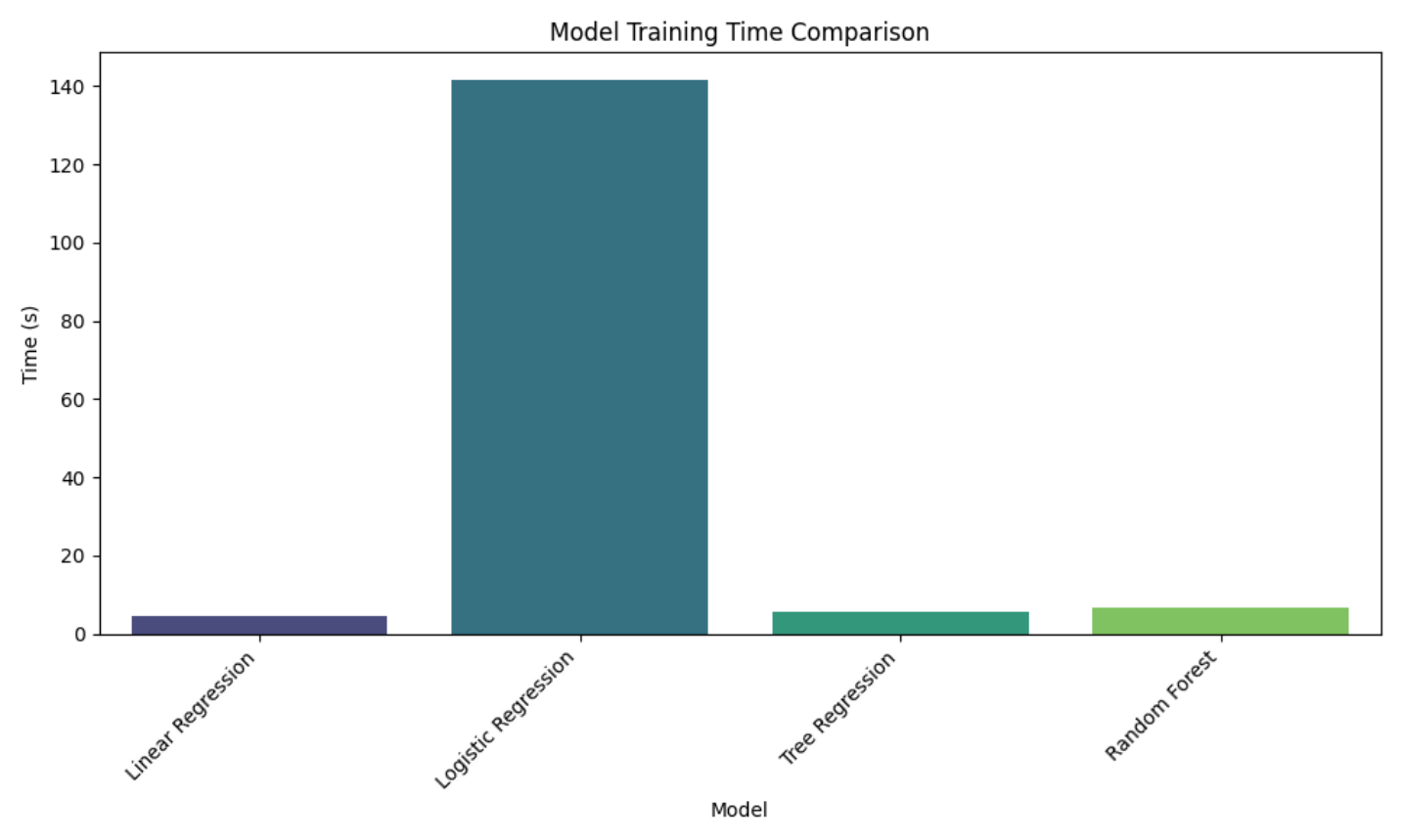
- Random projection is about 6 times faster than PCA
- The time complexity of PCA and random projection is similar since the data set is tall and skinny.
- The result is quite good since we decrease data size from more than 60 GB (csv file size) to several MB while reserve most of the feature in the data.

Regression

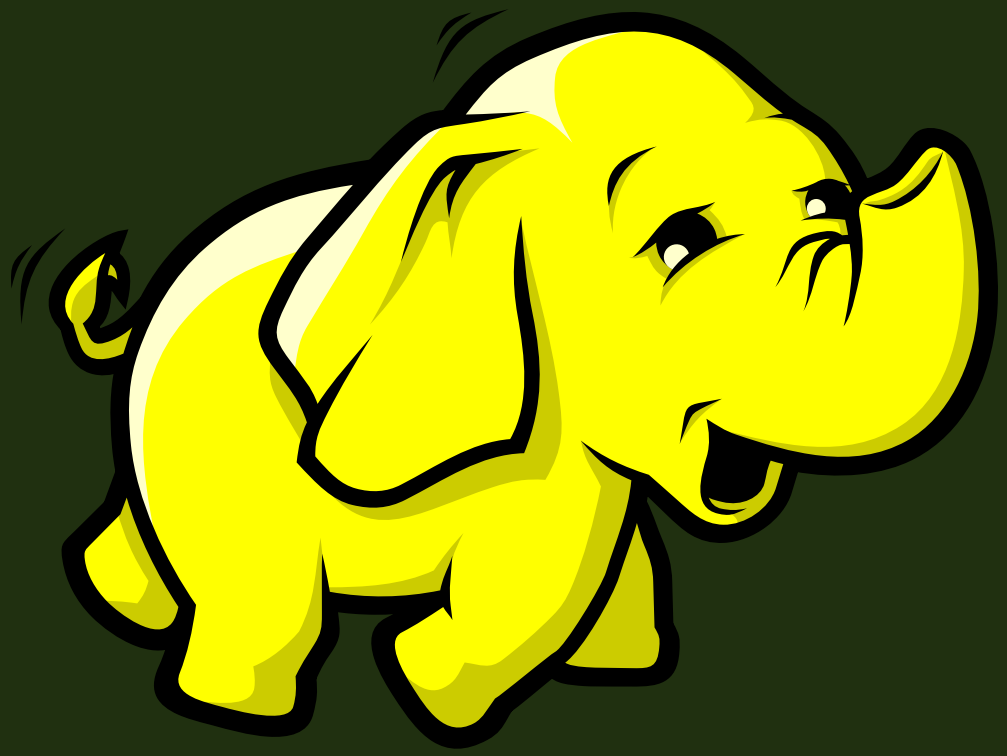
- In this part we test a wide range of regression methods on the processed values.
- We choose the linear regression, logistic regression and regression based on random forest.
- Since song style won't change in a short period of time, we also make every 10 year as label and predict based on that
- We didn't try to compare SGD and mini-batch since they are already replaced by L-BFGS.

Regression result

- We notice that labeling data make result enough worse, the accuracy decrease significantly, around 0.07
- Other regression method all have a RMSE around 10
- Among all methods linear regression is the fastest regression



- The accuracy caused by labeling possibly shows that the data in array type might have linear relation with year
- The same RMSE might show that we didn't include quantity in the data s.t. there is a loss we can't fix
- Since algorithm based on random forest can work in parallel: each tree doesn't matter another, it provides very fast speed and can use resource efficiently.
- Though 10 year loss is quite large, it still provides some information related to characteristic of specific era



Thank you!